

Day1

React js

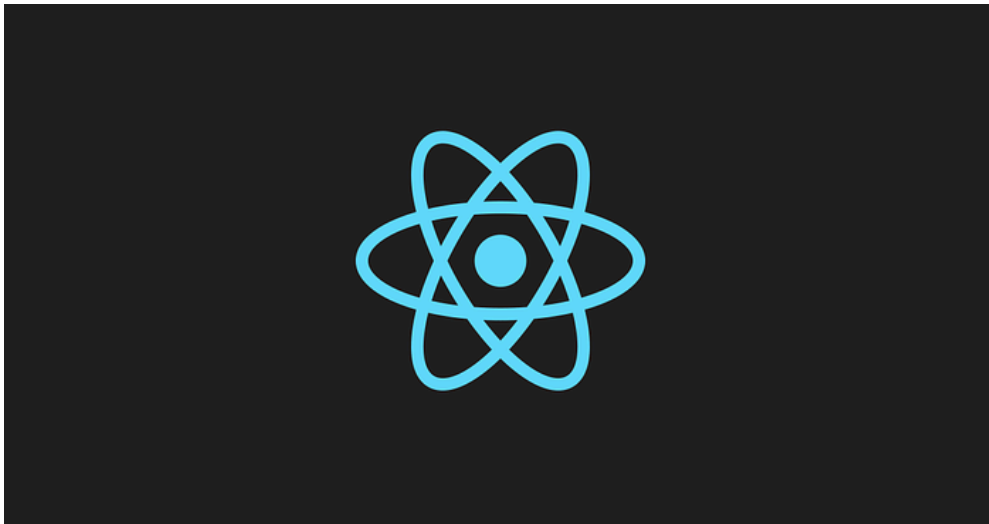
React is an open-source **JavaScript library** developed by Meta Platforms for building **declarative, component-based user interfaces (UIs)**, particularly for single-page applications (SPAs) and interactive web applications.

ReactJS: A brief history

React is a popular free and open-source front end Javascript library that is used for building UI components and user interfaces.

It was created by Facebook and is still maintained by them along with an ever growing community of companies and individual developers.

React is commonly used as the code base for SPA's ([Single Page Applications](#)).



But how did React get its start?

In 2011, as Facebook's Ad app began to pick up major momentum, their code began seeing a new updates and team members which became overwhelming and hard to manage after a while. In an attempt to help remedy this problem, [Jordan Walke](#) created [FaxJS](#), a prototype which made this process more efficient. This was the birth of what would later evolve into the version of React that we all currently know and love today.

In 2012, Facebook bought a little company called Instagram, who had just 13 employees at the time, for [\\$1 Billion](#). After its acquisition, Instagram wanted to adopt much of Facebook's new technology. This put pressure on the parent company to decouple React from Facebook and make it open-source, which is exactly what was done. In late May 2013, Jordan Walke finally introduced what was now called 'React' and open-sourced the material.

Some notable features which have helped make React so beloved by developers are its use of Components, the Virtual DOM, Lifecycle Methods, JSX, and React hooks.

React applications

Many popular websites and web applications are built with React due to its flexibility, performance, and scalability. Here are some well-known examples:

1. Facebook

- React was originally developed by Facebook, and it's used extensively across the platform. Both the Facebook website and the Instagram app use React in their user interfaces.

2. Instagram

- Instagram's web app is built using React, including features like the Explore page, photo/video uploads, and direct messaging.

3. Netflix

- Netflix uses React on its client-side for fast and smooth performance. The front-end architecture of their platform makes use of React for its user interface rendering.

4. Airbnb

- Airbnb uses React for its rich, interactive user experience. It powers many of their features, from search and filters to booking management.

5. WhatsApp Web

- The web version of WhatsApp, which offers messaging services directly from a browser, is built using React.

6. Uber

- Uber's web dashboard for managing rides and the admin panel for business management are built using React, providing a responsive and fluid experience.

7. Dropbox

- Dropbox uses React to manage its complex user interface and provide a seamless file-sharing experience.

8. BBC

- The British Broadcasting Corporation (BBC) uses React on its website to deliver content-rich pages with interactive elements and smooth navigation.

9. New York Times

- The New York Times uses React for interactive articles, multimedia content, and its crossword puzzle feature.

10. Atlassian Products (Jira, Confluence, Bitbucket)

- Atlassian's suite of products, including Jira, Confluence, and Bitbucket, use React to deliver rich, interactive user experiences for project management and collaboration.

11. GitHub

- GitHub's new UI, especially the code editor and notifications page, makes use of React to enhance performance and usability.

12. Reddit

- Reddit's redesign was done using React. The modern, responsive interface you see today is powered by React on both desktop and mobile versions.

13. Shopify

- Shopify uses React for its admin interface, providing a dynamic and interactive experience for shop owners managing their stores.

14. Trello

- Trello uses React for its web app, offering a smooth drag-and-drop interface for task and project management.

15. CodeSandbox

- CodeSandbox, an online code editor for web applications, is built using React. It allows developers to write, test, and share React code directly in the browser.

16. Discord

- Discord's web app uses React for rendering its user interface, allowing for real-time updates and seamless user interactions.

17. Salesforce

- Salesforce's Lightning Design System, used for building enterprise applications, is built with React.

18. Khan Academy

- Khan Academy uses React to create interactive and educational content, enabling a dynamic and responsive learning experience.

19. Pinterest

- Pinterest's web interface is built using React, ensuring fast and smooth interactions, especially when browsing through a large number of images.

20. Microsoft Products (Outlook, Teams)

- Microsoft has used React for its web versions of Outlook and Teams, providing a responsive and interactive user experience.

React: Component-Based Architecture, Virtual DOM, and SPA — Notes

1. Component-Based Architecture

Definition:

React applications are built using independent, reusable pieces called **components**. Each component encapsulates its own UI logic and presentation.

Why React Uses It:

- Promotes code reusability.
- Simplifies maintenance.
- Enables modular development.
- Improves scalability for large applications.
- Allows teams to work on different UI parts independently.

Example:

None

App

├─ Header

├─ Sidebar

├─ UserProfile

├─ Dashboard

└─ Footer

Each component can be developed, tested, and reused separately.

Benefits:

- Reusable code
- Easier debugging
- Better maintainability
- Faster development

- Separation of concerns
-

2. Virtual DOM

Definition:

The Virtual DOM is a lightweight JavaScript representation of the real browser DOM.

Why React Uses It:

Directly updating the browser DOM is expensive and can slow applications.

React:

1. Creates a Virtual DOM copy.
2. Updates the Virtual DOM when state changes.
3. Compares the new Virtual DOM with the previous one (**Diffing**).
4. Updates only the changed elements in the real DOM (**Reconciliation**).

Process:

None

State Change



Virtual DOM Updated



Diffing Algorithm



Find Changes



Update Real DOM Efficiently

Benefits:

- Faster UI updates
- Better performance
- Reduced DOM manipulation
- Improved user experience

Example:

Before:

HTML

```
<h1>Hello</h1>
```

After state change:

HTML

```
<h1>Hello John</h1>
```

React updates only the text node instead of re-rendering the entire page.

3. Single Page Application (SPA)

Definition:

A Single Page Application loads a single HTML page and dynamically updates content without full page reloads.

Traditional Multi-Page Application (MPA):

None

User Clicks Link



Request Sent



Server Generates New Page



Entire Page Reloads

React SPA:

None

User Clicks Link



React Router Changes View



Only Required Components Update



No Full Page Reload

Why React Is Popular for SPAs:

- Fast navigation
- Smooth user experience
- Reduced server load
- Efficient client-side rendering
- Better responsiveness

Examples:

- [Gmail](#)
- [Facebook](#)
- [Twitter/X](#)
- [Netflix](#)

React Project Setup

We use:

- Node.js
- Vite
- React

Step 1 – Open VS Code

Open terminal.

Step 2 – Create React Project

Shell

```
npm create vite@latest
```

Step 3 – Enter Project Name

None

doctorsapp

Step 4 – Select Framework

Choose:

None

React

Step 5 – Select Variant

Choose:

None

JavaScript

Step 6 – Move into Project

Shell

cd doctors-app

Step 7 – Install Packages

Shell

npm install

Step 8 – Run Project

Shell

npm run dev

Output

None

<http://localhost:5173>

Open in browser.

Detailed breakdown of the React + Vite project folder structure

React + Vite Project Folder Structure Explained

When you create a project with:

```
npm create vite@latest my-app -- --template react
```

You get this structure:

```
my-app/
├── node_modules/
├── public/
├── src/
│   ├── App.css
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── .gitignore
├── index.html
├── package.json
├── vite.config.js
└── README.md
```

Folder-by-Folder & File-by-File Explanation

Root Level Files & Folders

node_modules/

- Stores all installed npm packages
 - Auto-created after npm install
 - **Don't edit this folder!**
-

package.json

- Contains project metadata
- Lists all dependencies, scripts, and project name
- Example:

```
{
  "name": "my-app",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  },
  ...
}
```

vite.config.js

- Vite's configuration file
- You can add plugins, aliases, or environment settings here
- Basic config:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
});
```

index.html

- The main HTML file used in your app (not in /public)
- It contains the root `<div id="root">` where React renders
- Also links the entry script:

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

 **Vite directly uses this file**, unlike CRA.

.gitignore

- Tells Git which files/folders to ignore
 - Common ones:
 - node_modules/
 - dist/
 - .env
-

README.md

- Documentation file for your project
 - Great for giving an overview to other devs
-
-

public/

- Stores **static assets** (images, logos, etc.)
 - Files here are served **as-is**
 - Accessed via URL paths like /logo.png
 - **Avoid placing sensitive files here**
-

src/

Where all your **React code** lives. Let's break it down:

main.jsx

- **Entry point** of your React app
- Renders `<App />` into the root element

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';
```

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

App.jsx

- The **root component** of your app

- Acts like a layout or router wrapper

```
function App() {  
  return <h1>Hello Vite + React</h1>;  
}
```

```
export default App;
```

App.css and index.css

- Optional CSS files
- App.css for styling specific to the App component
- index.css for global styles (like resets, fonts, etc.)

✓ You can replace with Tailwind, CSS Modules, or styled-components if needed.

You can add more folders to src/:

Folder	Purpose
components/	Reusable components (like Button, Card)
pages/	Page-level components (Home, About)
assets/	Images, fonts, icons
hooks/	Custom React hooks
contexts/	Context providers (auth, theme)
services/	API functions or data fetching

!

JSX (JavaScript XML)

♦ **What is JSX?**

JSX stands for **JavaScript XML**. It is a **syntax extension** for JavaScript, commonly used with React to describe what the UI should look like. It looks like HTML but has the full power of JavaScript.

Why Use JSX?

- Makes code **easier to write and understand**.

- Looks like **HTML**, which is familiar to web developers.
 - Allows embedding **JavaScript expressions** within the markup.
 - JSX gets **compiled to React.createElement() calls** under the hood.
-

Babel

♦ What is Babel?

Babel is a **JavaScript compiler** that transforms modern JavaScript and JSX into code that browsers can understand.

JSX & Babel:

JSX is **not valid JavaScript**, so browsers can't read it directly. **Babel** converts JSX into plain JavaScript using the `React.createElement()` function.

Example:

Input (JSX):

```
const greeting = <h1>Hello, JSX!</h1>;
```

Babel Output:

```
const greeting = React.createElement('h1', null, 'Hello, JSX!');
```

So, Babel + JSX = Modern and browser-friendly code.

JSX Rules in React

JSX has **some strict rules and best practices**. Here's a full breakdown:

1. Return Only One Parent Element

A JSX expression must have **one root element**.

 Invalid:

```
const Parent = () => {  
  return (  
    <h1>Hello</h1>  
    <p>World</p>  
  );  
};
```

✓ Valid:

```
const Parent = () => {  
  return (  
    <div>  
      <h1>Hello</h1>  
      <p>World</p>  
    </div>  
  );  
};
```

2. Use className Instead of class

class is a reserved keyword in JavaScript.

✗ Invalid:

```
<div class="box"></div>
```

✓ Valid:

```
<div className="box"></div>
```

3. Use camelCase for Attributes

HTML attributes are written in **camelCase** in JSX.

✓ Examples:

```
<input type="text" />  
<button onClick={handleClick}>Click</button>
```

4. Close All Tags

All elements must be **self-closed** if they don't have children.

✗ Invalid:

```

```

✓ Valid:

```

```

```

```

5. JSX Allows JavaScript Expressions Inside {}

You can embed any valid JS expression using curly braces {}.

```
const name = "React";
return (
  <div>
    <h1>Hello, {name}</h1>
  </div>
);
```

6. No if Statements Inside JSX

You can't use if inside JSX directly – instead use:

- **Ternary operator** condition ? true : false
- **Logical AND** condition && expression

✓ Examples:

```
{isLoggedIn ? <h1>Welcome</h1> : <h1>Please log in</h1>}

{showMessage && <p>This is visible only when showMessage is true</p>}
```

7. Fragments (<>...</>) to Avoid Extra Nodes

React components must return one root element, but you can use **fragments** if you don't want to add extra HTML elements.

```
<>
  <h1>Title</h1>
  <p>Description</p>
</>
```

8. Inline Styling in JSX

Use style={{}} with camelCase properties.

```
<div style={{ backgroundColor: 'skyblue', padding: '10px' }}>Hello</div>
```

9. Comments in JSX

Use curly braces with regular JS comments:

```
{/* This is a comment */}
```

10. Component name should start with capital letter

React Components

♦ What is a Component?

- A **component** is a reusable bit of code responsible for rendering **part of the user interface (UI)**.
- It can be **as small as a button** or **as large as an entire page**.

♦ Types of Components

1. Functional Components

- o These are JavaScript functions that return JSX.
- o Simple and commonly used in modern React.

2. Class Components

- o Use ES6 classes.
- o Useful when working with older codebases or needing state/lifecycle methods (though now mostly replaced by hooks).

React Code- Day 1

Component Based Architecture

A React application is divided into small reusable pieces called Components.

Project Structure

```
Shell
doctorapp
|
├─ node_modules
├─ public
|
├─ src
|   |
|   ├─ assets
|   |
|   ├─ components
|   |   └─ Navbar.jsx
|   |   └─ Section.jsx
|   |   └─ styles.css
|   |
|   ├─ App.jsx
|   ├─ App.css
|   ├─ index.css
|   └─ main.jsx
|
├─ package.json
└─ vite.config.js
```

Components Created

Navbar Component

File

Shell
src/components/Navbar.jsx

Code

```
None
function Navbar() {
  return (
    <header>
      <h1>Doctor app</h1>

      <div>
        <button>Home</button>
        <button>Add doctor</button>
      </div>
    </header>
  );
}

export default Navbar;
```

Section Component

File

Shell

src/components/Section.jsx

Code

None

```
function Section() {  
  return (  
    <section className="s1">  
      <div>  
        <h1>Mr. Doctor app</h1>  
        <p>Find your doctors for your problems</p>  
      </div>  
  
      <div>  
          
      </div>  
    </section>  
  );  
}  
  
export default Section;
```

Root Component

File

Shell

src/App.jsx

Code

None

```
import Navbar from "../components/Navbar";
import Section from "../components/Section";
import "../components/styles.css";

function App() {
  return (
    <div>
      <Navbar />
      <Section />
    </div>
  );
}

export default App;
```

Styling

styles.css

Universal Selector

```
CSS
*{
    margin:0;
    padding:0;
    box-sizing:border-box;
}
```

Purpose

Removes default browser spacing.

Navbar Styling

```
CSS
header{
    display:flex;
    justify-content:space-between;
    align-items:center;
    background:blueviolet;
    padding:20px 30px;
    color:white;
    font-weight:900;
    font-size:2rem;
}
```

Button Styling

CSS

```
button{  
    padding:10px;  
    margin:5px;  
}
```

Section Styling

CSS

```
.s1{  
    display:flex;  
    justify-content:space-around;  
    align-items:center;  
    background:blueviolet;  
    color:white;  
    font-size:3rem;  
}
```

Child Div Styling

CSS

```
.s1 > div{  
    flex-basis:50%;  
    padding:30px;  
}
```

flex-basis

Determines how much space an item occupies inside a flex container.

None

50% = Half Width

Application Flow

None

`main.jsx`

↓

`App.jsx`

↓

`Navbar.jsx`

↓

`Section.jsx`

React starts rendering from `main.jsx`.

Output

Doctor app

[Home](#)[Add doctor](#)

Mr. Doctor app

Find your doctors for your problems

